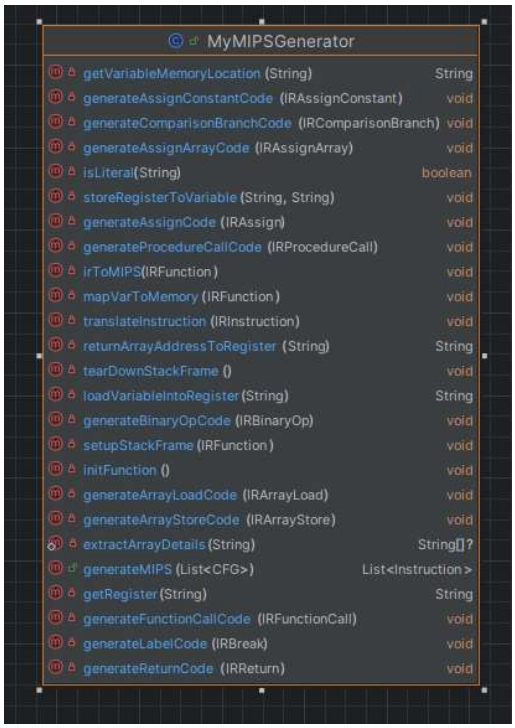
Phase 3 Internal Design:

1. MIPS Function Calling Convention Implementation:
    a. MIPS Generator perform the functions of translating the IR representation to the assembly code. The generator uses static data from IRInstruction from phase 2 to initialize the .data section. Once that done, the generator class with analysis the function data for total size needed for stack frame, including the max number of arguments when passing on the stack, the mapping of local variables to the stack area is also done in this step. The general layout of the stack from is:
        - 2 word for frame pointer and return address
        - 8 word for saved register
        - Spill location for local variable/array
        - 4 – max arguments for function arguments passed on the stack

        For the naïve approach, correctly mapping the variables to memory address is especially important since each variable are load/store on every instruction.



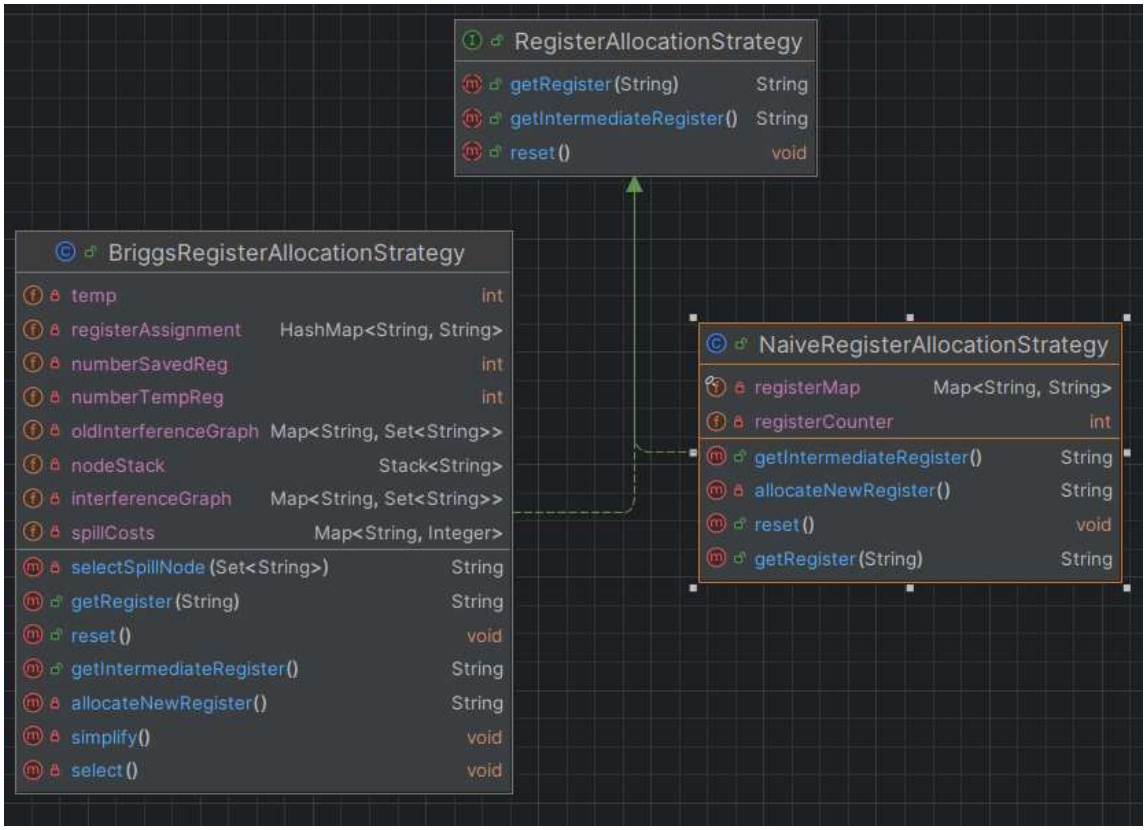
    b. Each IRInstruction is converted to MIPS instruction. There are some op in IR instruction needs to use multiple MIPS instructions to complete. One such instruction is the Array Assign instruction in which the whole array is copy element by element in MIPS instruction. In my design, this was done through a loop by loading in the base address of the arrays and shifting them by 4 while load/store the value.

2. MIPSGenerator employs two strategies to allocate register: Naïve and Briggs.

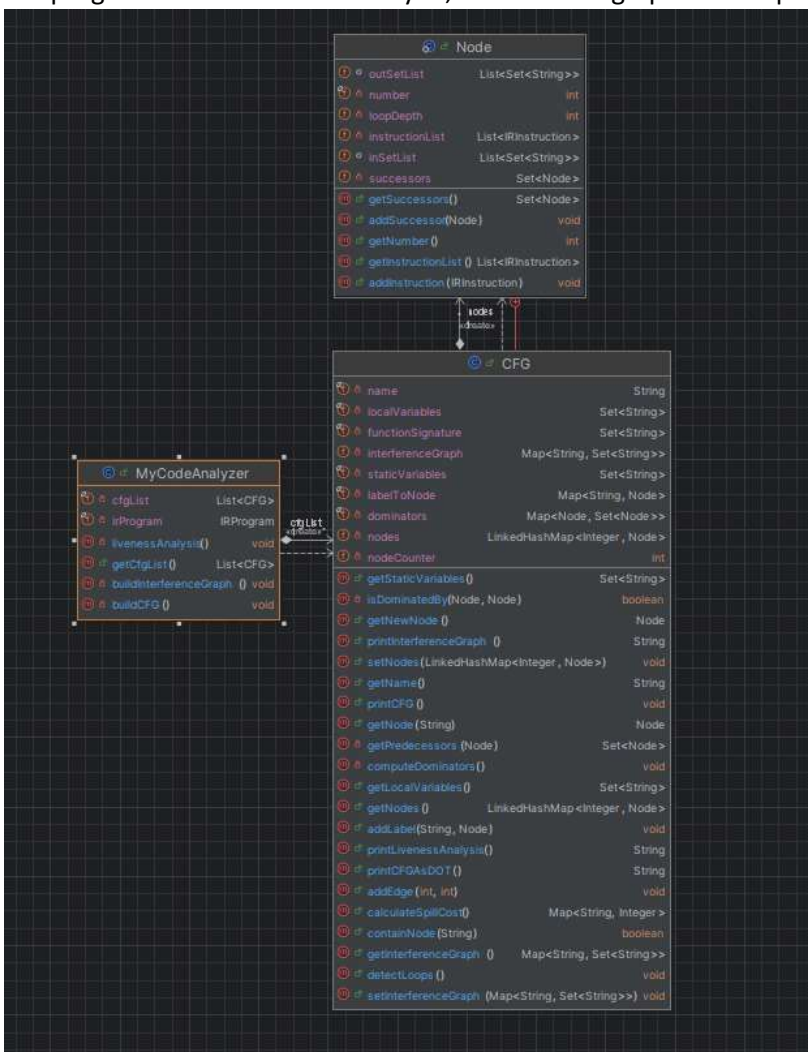
   The two classes implement the RegisterAllocationStrategy so that they can provide the same calling method to MIPSGenerator. The two classes strictly deal with returning available register to MIPSGenerator. The logic on whether the value needed to be loaded into the register or store back to memory are strictly handle by the MIPSGenerator.

   a. Briggs: Briggs algorithm depends on the interference graph. We represent the graph as an adjacency list. By iterate through the list, we can find each Node that has less than K number of edges and remove it. I believe the time complexity is O(mn) due to the fact that we are also transversing and remove all edge connected to the node. After transferring to the node stack, we start rebuilding the graph. Using an old graph that was deep copied, we can take the node on top the stack and give it a color. All node that's follow will have to check with its neighboring of what color they have. If there is not color available left, then we will have to spill the node.

   b. Naïve approach returns a temporary register when query with getRegister() method. Naïve method does not keep track of any register to variable mapping. MIPSGenerator is solely responsible for determining which variable needs to be store back and which variable needs to be loaded. Generally, variable from use list will need to be load while variable on def list will need to be loaded back to memory location.



3. Code Analysis:
    a. CFG is the class object that hole the code analysis for each function. Node is the subclass that hold information for each basic block.
    b. With code analysis, we start from the small granularity of each individual instruction, determined the inset/outset list. Then move on with finding the bigger basic block to generate the control flow graph. Using the control flow graph, we can determine the overall data flow of the program such as liveness analysis, interference graph and loop detection analysis.



4. Bug/Non Functional features
   Currently Briggs allocation method isn't handling the register correct. This might be an issue with the actually implementation of the algorithm. Even when restricting to only use the saved register and save/load register across function call, we're still getting incorrect value load back into the register.